

Java Collection Framework Solutions Handbook

Think-Pair-Share | Continuous Internal Evaluation (CIE)

20 Problems with Complete Java Solutions
Second Year B.Tech | Java Programming

Covers: ArrayList · LinkedList · HashMap · HashSet · LinkedHashMap · ArrayDeque · TreeMap · Nested
Collections

Level 1 — Beginner Problems (01–07)

#01 Student Marks Register

Collection: ArrayList<Integer> **Industry:** Education **Level:** Beginner

Scenario: A school teacher stores marks of all students, finds the highest mark, lowest mark, and calculates the class average.

Java Solution:

```
import java.util.*;

public class StudentMarksRegister {
    public static void main(String[] args) {
        ArrayList<Integer> marks = new ArrayList<>();
        // Add 10 student marks
        int[] raw = {85, 92, 78, 95, 60, 74, 88, 55, 91, 70};
        for (int m : raw) marks.add(m);

        System.out.println("All Marks: " + marks);
        System.out.println("Highest Mark : " + Collections.max(marks));
        System.out.println("Lowest Mark : " + Collections.min(marks));

        int sum = 0;
        for (int m : marks) sum += m;
        double avg = (double) sum / marks.size();
        System.out.printf("Class Average: %.2f%n", avg);
    }
}
```

#02 Shopping Cart Management

Collection: ArrayList<String> **Industry:** E-Commerce **Level:** Beginner

Scenario: Basic shopping cart: add items, remove an item by name, and view cart before checkout.

Java Solution:

```
import java.util.*;

public class ShoppingCart {
    private ArrayList<String> cart = new ArrayList<>();

    public void addItem(String name) { cart.add(name); }
    public void removeItem(String name) {
        if (cart.contains(name)) cart.remove(name);
        else System.out.println(name + " not found in cart.");
    }
    public void displayCart() {
        System.out.println("Cart (" + cart.size() + " items): " + cart);
    }

    public static void main(String[] args) {
        ShoppingCart sc = new ShoppingCart();
        sc.addItem("Laptop"); sc.addItem("Mouse"); sc.addItem("Keyboard");
        sc.addItem("Headphones"); sc.addItem("Webcam");
        sc.displayCart();
        sc.removeItem("Mouse");
        sc.displayCart();
    }
}
```

#03 Employee Attendance Tracker

Collection: ArrayList<String> **Industry:** HR / Corporate **Level:** Beginner

Scenario: Track which employees are present each day; check presence and count total attendance.

Java Solution:

```
import java.util.*;

public class AttendanceTracker {
    public static void main(String[] args) {
        ArrayList<String> present = new ArrayList<>();
        String[] employees = {"Alice", "Bob", "Carol", "David",
                               "Eve", "Frank", "Grace", "Hank"};
        for (String e : employees) present.add(e);

        System.out.println("Attendance List: " + present);
        System.out.println("Total Present : " + present.size());

        String check = "Carol";
        System.out.println(check + " present? " + present.contains(check));
        System.out.println("Index of Bob : " + present.indexOf("Bob"));
    }
}
```

#04 Music Player Queue

Collection: LinkedList<String> **Industry:** Music Streaming **Level:** Beginner

Scenario: Song queue: add songs to end/front, play (remove) next song.

Java Solution:

```
import java.util.*;

public class MusicPlayerQueue {
    private LinkedList<String> queue = new LinkedList<>();

    public void addSongToEnd(String title) { queue.addLast(title); }
    public void addUrgentSong(String title) { queue.addFirst(title); }
    public void playNextSong() {
        if (queue.isEmpty()) { System.out.println("Queue empty!"); return; }
        System.out.println("Now Playing: " + queue.removeFirst());
    }

    public static void main(String[] args) {
        MusicPlayerQueue mp = new MusicPlayerQueue();
        mp.addSongToEnd("Blinding Lights");
        mp.addSongToEnd("Shape of You");
        mp.addSongToEnd("Stay");
        System.out.println("Queue: " + mp.queue);
        mp.addUrgentSong("Levitating");
        System.out.println("After urgent add: " + mp.queue);
        mp.playNextSong(); mp.playNextSong(); mp.playNextSong();
        System.out.println("Remaining: " + mp.queue);
    }
}
```

#05 Student Grade Lookup

Collection: HashMap<String, String> **Industry:** Education **Level:** Beginner

Scenario: Instantly look up the grade of any student by roll number; handle missing roll numbers.

Java Solution:

```
import java.util.*;

public class StudentGradeLookup {
    public static void main(String[] args) {
        HashMap<String,String> grades = new HashMap<>();
        grades.put("R001","A+"); grades.put("R002","B");
        grades.put("R003","A"); grades.put("R004","C+");
        grades.put("R005","B+"); grades.put("R006","A-");

        System.out.println("Grade of R003 : " + grades.get("R003"));
        System.out.println("Grade of R999 : " +
            grades.getOrDefault("R999","Not Found"));
        System.out.println("Contains R004? " + grades.containsKey("R004"));

        System.out.println("\nAll Grades:");
        for (Map.Entry<String,String> e : grades.entrySet())
            System.out.println("  " + e.getKey() + " -> " + e.getValue());
    }
}
```

#06 ATM Balance Checker

Collection: HashMap<String, Double> **Industry:** Banking **Level:** Beginner

Scenario: Check balances, credit (deposit), and debit (withdraw) for customers by account number.

Java Solution:

```
import java.util.*;

public class ATMBalanceChecker {
    private HashMap<String,Double> accounts = new HashMap<>();

    public void deposit(String acc, double amt) {
        accounts.put(acc, accounts.getOrDefault(acc, 0.0) + amt);
        System.out.printf("Deposited %.2f -> %s Balance: %.2f\n",
            amt, acc, accounts.get(acc));
    }

    public void withdraw(String acc, double amt) {
        if (!accounts.containsKey(acc)) { System.out.println("Account not found."); return; }
        if (accounts.get(acc) < amt) { System.out.println("Insufficient balance."); return; }
        accounts.put(acc, accounts.get(acc) - amt);
        System.out.printf("Withdrawn %.2f <- %s Balance: %.2f\n",
            amt, acc, accounts.get(acc));
    }

    public void getBalance(String acc) {
        System.out.println("Balance[" + acc + "] = " +
            accounts.getOrDefault(acc, 0.0));
    }

    public static void main(String[] args) {
        ATMBalanceChecker atm = new ATMBalanceChecker();
        atm.deposit("ACC01",5000); atm.deposit("ACC02",12000);
        atm.deposit("ACC03",3000); atm.deposit("ACC04",8500);
        atm.withdraw("ACC01",1500); atm.withdraw("ACC03",5000);
        atm.getBalance("ACC02");
    }
}
```

#07 Duplicate Voter ID Detector

Collection: HashSet<String> **Industry:** Government / Election **Level:** Beginner

Scenario: Ensure no voter casts their vote twice; reject duplicate voter ID submissions.

Java Solution:

```
import java.util.*;

public class VoterIDDetector {
    public static void main(String[] args) {
        HashSet<String> used = new HashSet<>();
        String[] incoming = {"V001", "V002", "V003", "V001", "V004",
                             "V005", "V002", "V006", "V007", "V003"};
        for (String id : incoming) {
            if (used.add(id)) System.out.println(id + " -> Vote Accepted");
            else               System.out.println(id + " -> Duplicate vote rejected!");
        }
        System.out.println("\nTotal valid votes: " + used.size());
    }
}
```

Level 2 — Easy Problems (08–15)

#08 Supermarket Billing System

Collection: ArrayList<Double> **Industry:** Retail **Level:** Easy

Scenario: Store item prices, calculate subtotal, apply 10% discount if bill exceeds Rs.500.

Java Solution:

```
import java.util.*;

public class SupermarketBilling {
    public static void main(String[] args) {
        ArrayList<Double> prices = new ArrayList<>()
            Arrays.asList(120.0, 85.5, 200.0, 45.0, 99.9, 60.0));

        Collections.sort(prices);
        System.out.println("Itemised Bill (sorted): " + prices);

        double subtotal = 0;
        for (double p : prices) subtotal += p;

        double discount = (subtotal > 500) ? subtotal * 0.10 : 0;
        double total = subtotal - discount;

        System.out.printf("Subtotal : Rs. %.2f\n", subtotal);
        System.out.printf("Discount : Rs. %.2f\n", discount);
        System.out.printf("Payable : Rs. %.2f\n", total);
    }
}
```

#09 Customer Service Ticket Queue

Collection: LinkedList<String> **Industry:** IT Support / Customer Care **Level:** Easy

Scenario: FIFO ticket queue: new tickets join end, agents process from front.

Java Solution:

```
import java.util.*;

public class TicketQueue {
```

```

public static void main(String[] args) {
    LinkedList<String> queue = new LinkedList<>();

    // 5 tickets arrive
    for (int i = 1; i <= 5; i++) { queue.offer("TKT-00" + i); }
    System.out.println("Initial Queue: " + queue);

    // Agent processes 2 tickets
    System.out.println("Processing: " + queue.poll());
    System.out.println("Processing: " + queue.poll());
    System.out.println("Queue after 2 processed: " + queue);

    // 2 more arrive
    queue.offer("TKT-006"); queue.offer("TKT-007");
    System.out.println("Queue after 2 new arrivals: " + queue);
    System.out.println("Next to process (peek): " + queue.peek());
}
}

```

#10 Browser Back Button

Collection: ArrayDeque<String> as Stack **Industry:** Web Browser **Level:** Easy

Scenario: Every visited URL is pushed; Back button pops and shows previous page.

Java Solution:

```

import java.util.*;

public class BrowserHistory {
    private ArrayDeque<String> history = new ArrayDeque<>();

    public void visitPage(String url) {
        history.push(url);
        System.out.println("Visited: " + url);
    }

    public void goBack() {
        if (history.isEmpty()) { System.out.println("No history!"); return; }
        System.out.println("Back from: " + history.pop());
        System.out.println("Now at    : " + (history.isEmpty()?"(start)":history.peek()));
    }

    public void getCurrentPage() {
        System.out.println("Current  : " + (history.isEmpty()?"(start)":history.peek()));
    }

    public static void main(String[] args) {
        BrowserHistory b = new BrowserHistory();
        b.visitPage("google.com"); b.visitPage("stackoverflow.com");
        b.visitPage("github.com"); b.visitPage("oracle.com");
        b.visitPage("youtube.com");
        b.getCurrentPage();
        b.goBack(); b.goBack();
        b.getCurrentPage();
    }
}

```

#11 Parenthesis Checker for Code Editor

Collection: ArrayDeque<Character> as Stack **Industry:** IDE / Developer Tools **Level:** Easy

Scenario: Validate if brackets in code are balanced using a stack.

Java Solution:

```

import java.util.*;

public class ParenthesisChecker {
    public static boolean isValid(String s) {
        ArrayDeque<Character> stack = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if (c=='(' || c=='[' || c=='{') { stack.push(c); }
            else if (c==')' || c==']' || c=='}') {
                if (stack.isEmpty()) return false;
                char top = stack.pop();
                if ((c==')' && top!='(') ||
                    (c==']' && top!='[') ||
                    (c=='}' && top!='{')) return false;
            }
        }
        return stack.isEmpty();
    }

    public static void main(String[] args) {
        String[] tests = {
            "{[()]}", // valid
            "([)]", // invalid
            "((()))", // valid
            "{[]", // invalid
            "" // valid (empty)
        };
        for (String t : tests)
            System.out.println("'" + t + "' -> " +
                               (isValid(t) ? "Valid" : "Invalid"));
    }
}

```

#12 Ticket Booking Counter

Collection: ArrayDeque<String> as Queue **Industry:** Railways / Cinema **Level:** Easy

Scenario: Railway ticket counter serves passengers FIFO; display who is being served.

Java Solution:

```

import java.util.*;

public class BookingCounter {
    public static void main(String[] args) {
        ArrayDeque<String> queue = new ArrayDeque<>();
        String[] initial = {"Amit", "Priya", "Ravi", "Sneha", "Kiran", "Meera"};
        for (String p : initial) queue.offer(p);
        System.out.println("Initial Queue: " + queue);

        // Serve first 3
        for (int i = 0; i < 3; i++)
            System.out.println("Now serving: " + queue.poll() +
                               " | Remaining: " + queue);

        // 2 more join
        queue.offer("Deepak"); queue.offer("Neha");
        System.out.println("After 2 new arrivals: " + queue);

        // Serve rest
        while (!queue.isEmpty())
            System.out.println("Now serving: " + queue.poll());
    }
}

```

#13 Unique Product Tag Filter

Collection: HashSet<String> **Industry:** E-Commerce **Level:** Easy

Scenario: Store only unique product tags; find common tags between two products.

Java Solution:

```
import java.util.*;

public class ProductTagFilter {
    public static void main(String[] args) {
        // Product 1 tags (with duplicates)
        String[] raw1 = {"electronics","sale","new","gadget","sale",
                        "electronics","portable","wireless","new","budget"};
        HashSet<String> tags1 = new HashSet<>(Arrays.asList(raw1));
        System.out.println("Product 1 Unique Tags (" + tags1.size() + "): " + tags1);

        // Product 2 tags
        HashSet<String> tags2 = new HashSet<>(
            Arrays.asList("wireless","audio","portable","premium","new"));
        System.out.println("Product 2 Tags: " + tags2);

        // Intersection
        HashSet<String> common = new HashSet<>(tags1);
        common.retainAll(tags2);
        System.out.println("Common Tags: " + common);

        // Union
        HashSet<String> union = new HashSet<>(tags1);
        union.addAll(tags2);
        System.out.println("All Tags      : " + union);
    }
}
```

#14 Word Frequency Counter

Collection: HashMap<String, Integer> **Industry:** Search / Analytics **Level:** Easy

Scenario: Count how many times each word appears in a user review; display sorted by frequency.

Java Solution:

```
import java.util.*;

public class WordFrequencyCounter {
    public static void main(String[] args) {
        String sentence = "java is great and java is fast and java collections are great";
        String[] words = sentence.replaceAll("[^a-zA-Z ]","").toLowerCase().split("\\s+");

        HashMap<String,Integer> freq = new HashMap<>();
        for (String w : words)
            freq.put(w, freq.getOrDefault(w, 0) + 1);

        // Sort by frequency descending
        List<Map.Entry<String,Integer>> list = new ArrayList<>(freq.entrySet());
        list.sort((a,b) -> b.getValue() - a.getValue());

        System.out.println("Word Frequencies (descending):");
        for (Map.Entry<String,Integer> e : list)
            System.out.printf(" %-15s : %d\n", e.getKey(), e.getValue());
    }
}
```


#15 Recently Viewed Products

Collection: LinkedHashSet<String> **Industry:** E-Commerce **Level:** Easy

Scenario: Show last 5 uniquely viewed products in order; revisited product moves to most recent position.

Java Solution:

```
import java.util.*;

public class RecentlyViewed {
    private static final int MAX = 5;
    private LinkedHashSet<String> history = new LinkedHashSet<>();

    public void viewProduct(String name) {
        history.remove(name); // remove if already present
        history.add(name);    // add to end (most recent)
        if (history.size() > MAX) {
            String oldest = history.iterator().next();
            history.remove(oldest);
        }
        System.out.println("Viewed: " + name + " | History: " + history);
    }

    public static void main(String[] args) {
        RecentlyViewed rv = new RecentlyViewed();
        String[] views = {"Phone", "Laptop", "Headphones", "Tablet", "Watch",
                          "Phone", "Charger", "Laptop"};
        for (String p : views) rv.viewProduct(p);
    }
}
```

Level 3 — Easy+ Problems (16–20)

#16 Student Rank List (Sorted by Marks)

Collection: TreeMap<Integer, String> **Industry:** Education **Level:** Easy+

Scenario: Automatically sorted rank list; display highest to lowest marks, find topper and weakest.

Java Solution:

```
import java.util.*;

public class StudentRankList {
    public static void main(String[] args) {
        TreeMap<Integer, String> rankMap = new TreeMap<>();
        rankMap.put(88, "Alice"); rankMap.put(95, "Bob");
        rankMap.put(72, "Carol"); rankMap.put(60, "David");
        rankMap.put(91, "Eve");   rankMap.put(78, "Frank");

        System.out.println("Rank List (Ascending by marks):");
        for (Map.Entry<Integer, String> e : rankMap.entrySet())
            System.out.println(" " + e.getValue() + " -> " + e.getKey());

        System.out.println("\nRank List (Descending - Top to Bottom):");
        for (Map.Entry<Integer, String> e : rankMap.descendingMap().entrySet())
            System.out.println(" " + e.getValue() + " -> " + e.getKey());

        System.out.println("\nTopper : " + rankMap.get(rankMap.lastKey())
                           + " (" + rankMap.lastKey() + ")");
        System.out.println("Weakest : " + rankMap.get(rankMap.firstKey())
                           + " (" + rankMap.firstKey() + ")");
    }
}
```

```
}
```

#17 Phone Directory (Alphabetical)

Collection: TreeMap<String, String> **Industry:** Telecom / Contacts App **Level:** Easy+

Scenario: Contacts always displayed alphabetically; search by name; get contacts between two names.

Java Solution:

```
import java.util.*;

public class PhoneDirectory {
    public static void main(String[] args) {
        TreeMap<String,String> dir = new TreeMap<>();
        dir.put("Alice","9876543210");  dir.put("Bob","8765432109");
        dir.put("Carol","7654321098");  dir.put("David","6543210987");
        dir.put("Eve","5432109876");    dir.put("Frank","4321098765");
        dir.put("Grace","3210987654");  dir.put("Hank","2109876543");

        System.out.println("All Contacts (Alphabetical):");
        for (Map.Entry<String,String> e : dir.entrySet())
            System.out.println(" " + e.getKey() + " : " + e.getValue());

        System.out.println("\nSearch Carol : " + dir.get("Carol"));
        System.out.println("First contact : " + dir.firstKey());
        System.out.println("Last contact : " + dir.lastKey());

        System.out.println("\nContacts between Bob and Frank:");
        for (Map.Entry<String,String> e : dir.subMap("Bob",true,"Frank",true).entrySet())
            System.out.println(" " + e.getKey() + " : " + e.getValue());
    }
}
```

#18 Subject-wise Student Groups

Collection: HashMap<String, ArrayList<String>> **Industry:** Education **Level:** Easy+

Scenario: Group students subject-wise; add students, list subjects, find subjects of a student.

Java Solution:

```
import java.util.*;

public class SubjectGroups {
    private HashMap<String,ArrayList<String>> groups = new HashMap<>();

    public void addStudent(String subject, String name) {
        groups.computeIfAbsent(subject, k -> new ArrayList<>()).add(name);
    }
    public List<String> getStudents(String subject) {
        return groups.getOrDefault(subject, new ArrayList<>());
    }
    public List<String> findSubjectsOf(String student) {
        List<String> result = new ArrayList<>();
        for (Map.Entry<String,ArrayList<String>> e : groups.entrySet())
            if (e.getValue().contains(student)) result.add(e.getKey());
        return result;
    }

    public static void main(String[] args) {
        SubjectGroups sg = new SubjectGroups();
        sg.addStudent("Java","Alice"); sg.addStudent("Java","Bob");
        sg.addStudent("Java","Carol");
    }
}
```

```

        sg.addStudent("Python", "Alice"); sg.addStudent("Python", "David");
        sg.addStudent("Python", "Eve");
        sg.addStudent("DBMS", "Bob"); sg.addStudent("DBMS", "Carol");
        sg.addStudent("DBMS", "Frank");

        System.out.println("Java students : " + sg.getStudents("Java"));
        System.out.println("Python students: " + sg.getStudents("Python"));
        System.out.println("Subjects of Alice: " + sg.findSubjectsOf("Alice"));
        System.out.println("Subjects of Bob : " + sg.findSubjectsOf("Bob"));
    }
}

```

#19 Undo Feature in Text Editor

Collection: ArrayDeque<String> as Stack **Industry:** Productivity Apps **Level:** Easy+

Scenario: Save document state after every change; Ctrl+Z restores previous state using a stack.

Java Solution:

```

import java.util.*;

public class TextEditorUndo {
    private ArrayDeque<String> undoStack = new ArrayDeque<>();
    private String current = "";

    public void type(String text) {
        undoStack.push(current);
        current += text;
        System.out.println("Typed : " + text + " | Doc: " + current + "");
    }

    public void undo() {
        if (undoStack.isEmpty()) { System.out.println("Nothing to undo!"); return; }
        current = undoStack.pop();
        System.out.println("Undo! Doc restored to: " + current + "");
    }

    public static void main(String[] args) {
        TextEditorUndo ed = new TextEditorUndo();
        ed.type("Hello ");
        ed.type("World ");
        ed.type("from ");
        ed.type("Java ");
        ed.type("Collections!");
        System.out.println("\n--- Pressing Undo 3 times ---");
        ed.undo(); ed.undo(); ed.undo();
        System.out.println("Final state: " + ed.current + "");
        ed.undo(); ed.undo(); ed.undo(); // extra undos
    }
}

```

#20 Order Processing System

Collection: ArrayDeque<String> as Queue **Industry:** Food Delivery / Logistics **Level:** Easy+

Scenario: Process customer orders in FIFO order; kitchen dispatches from front, new orders join end.

Java Solution:

```

import java.util.*;

public class OrderProcessingSystem {
    public static void main(String[] args) {
        ArrayDeque<String> orders = new ArrayDeque<>();
    }
}

```

```

// 6 orders arrive
for (int i = 1; i <= 6; i++) orders.offer("Order-" + i);
System.out.println("Initial Orders: " + orders);

int cycle = 1;
while (!orders.isEmpty()) {
    System.out.println("\n-- Cycle " + cycle++ + " --");
    System.out.println("Processing: " + orders.poll());
    System.out.println("Queue      : " + orders);
    if (cycle == 3) {
        orders.offer("Order-7"); orders.offer("Order-8");
        System.out.println("2 new orders added: " + orders);
    }
}
System.out.println("\nAll orders processed!");
}
}

```